

PRISM: Local-First Smart Contract Auditing with Graph Screening, Open-Weight LLMs and EVM Validation

Anonymous Author(s)

Anonymous Institution

Abstract. Auditing unreleased smart contracts with hosted large language models (LLMs) discloses proprietary source code and business logic to the model provider. We present PRISM, an audit pipeline in which every learned stage runs on the audit host. PRISM removes secrets and reversibly pseudonymizes identifiers, converts Slither’s static single assignment (SSA) intermediate representation into typed function-level code property graphs (CPGs), routes functions with a two-stage graph attention classifier, queries a locally served open-weight code LLM for vulnerability hypotheses, and checks invariant harnesses with Foundry on a local Ethereum Virtual Machine (EVM). On 12,991 public function graphs covering five Smart Contract Weakness Classification (SWC) classes, the classifier reaches $75.95 \pm 0.19\%$ macro-F1 on a random split and $76.31 \pm 0.06\%$ on a lineage-disjoint split over five seeds, at 0.21 ms per function. Graph convolutional and graph isomorphism networks reach the same level, share at least 97% of its errors and differ in none of 20 paired tests, which places the remaining error in contract-derived labels rather than in the encoder. On 57 benchmark contracts, pseudonymization preserves compilation and every Slither detector type, yet a frequency-aware adversary re-links 89.5% of identifiers, so confidentiality has to come from local inference rather than from masking. Static filtering reduces flagged safe contracts from 27/27 to 6/27 while retaining 18 of 30 vulnerable contracts, and 300 seeded Foundry runs on reference harnesses complete deterministically.

Keywords: Smart contract security · Confidential auditing · Graph neural networks · Local large language models · Pseudonymization.

1 Introduction

Smart contracts on the Ethereum Virtual Machine (EVM) hold large amounts of value in decentralized finance (DeFi), and deployed bytecode cannot be patched in place [2, 18]. Flaws must therefore be found before deployment, usually by auditing source code that its owner has not yet released and considers proprietary [2].

Large language models (LLMs) are now the most active direction in automated auditing: LLM agents audit with fine-tuned detectors or in multi-agent teams [19,

30], pair with formal verification or program analysis [16, 26], and derive seeds for fuzzing [10, 15]. Graph learning has advanced in parallel, up to heterogeneous graph transformers with LLM-derived features [20]. Classical tools explain why learning is attractive: 97% of 47,587 contracts were flagged by at least one static analyzer [7].

Most LLM auditors call hosted models [16, 26, 30], so the auditor transmits unreleased code, identifier names and business logic to a model provider. Provider-side confidential inference remains a research prototype [14], and open-weight detectors [34] return labels without executable evidence. Masking identifiers before a hosted call is the obvious alternative, but prompt rewriting for code assistants still leaks [23], and learned models recover variable names from stripped binaries [32, 33]. An auditor who must keep code in house therefore faces two open questions: which parts of an LLM-assisted audit can run on commodity local hardware, and whether cheap pseudonymization would make hosted models acceptable after all.

PRISM builds on the observation that the steps of an LLM-assisted audit need different guarantees. Deciding where to look is a routing problem that a compact graph model solves in well under a millisecond per function. Proposing what may be wrong benefits from a code LLM but does not need a frontier model when its output is treated as a hypothesis. Deciding what is real requires execution. PRISM therefore places a two-stage graph attention classifier in front of a locally served open-weight code LLM and a Foundry execution back end, keeps every model and artifact on the audit host, pseudonymizes identifiers before any learned stage and restores them only in the final report. Our contributions are:

- A local-first audit pipeline with an explicit two-adversary threat model, in which every stage runs on the audit host and LLM hypotheses are kept separate from execution-confirmed findings (Sect. 3).
- A function-level code property graph (CPG) over Slither’s static single assignment (SSA) form with typed control- and data-flow edges, and a two-stage graph attention classifier that routes functions at $76.31 \pm 0.06\%$ macro-F1 on unseen contract lineages; the generic encoders we test share at least 97% of its errors, and a label-conflict diagnostic places the remaining error in contract-derived labels (Sect. 5.1).
- A measurement of what reversible pseudonymization preserves and leaks: masked code keeps compilation and all 229 Slither detector types, whereas a frequency adversary re-links 89.5% of identifiers and masking lowers unguided local LLM recall from 30/30 to 13/30 (Sect. 5.3).
- A scoped evaluation of the static and execution stages on 57 contracts: filtering clears 21 of 27 flagged safe contracts, 300 seeded Foundry runs complete deterministically, and five template patches pass five validation checks (Sects. 5.2–5.5).

The rest of the paper is organized as follows: Sect. 2 reviews related work, Sects. 3–5 present design, setup and results, Sect. 6 discusses the scope of the evidence, and Sect. 7 concludes.

2 Related Work

Static analysis and fuzzing. Oyente [18] applies symbolic execution, Securify [27] checks Datalog patterns, and Slither [8] runs rule-based detectors on SlithIR, an SSA intermediate representation (IR); empirical studies and surveys [7, 11, 36] report low precision and uneven recall for such tools. Fuzzers contribute call-level oracles [13], property-based [12], adaptive [22], greybox [31], data-flow-guided [4] and snapshot-based [25] exploration, but still need guidance to pass narrow guards.

Graph learning and label quality. A recent survey [1] and early models [17, 38] learn over contract graphs with normalized nodes and expert patterns, DA-GNN [35] applies dual attention, ConSense [24] combines local and global attention, and MANDO-LLM [20] feeds LLM code features into heterogeneous graph transformers. PRISM’s classifier builds on graph attention networks (GAT) and GATv2 [3, 28] over a homogeneous SSA-level function graph with typed edge attributes; its role is to decide which functions later stages examine, not to issue the final verdict. Studies of general vulnerability datasets show that noisy and inherited labels inflate or cap reported accuracy [5, 6], which is the effect we measure for contract-derived labels.

LLMs for auditing, fuzzing and repair. GPTScan [26] pairs GPT with static confirmation, PropertyGPT [16] and SmartInv [29] generate properties or invariants for verification, LLM-SmartAudit [30] and iAudit [19] organize LLM agents around the audit, and PromFuzz [15] and LLAMA [10] guide fuzzing.

Table 1. Capability comparison, sorted by year. GNN: graph detector; LLM: uses an LLM; Local: model runs on premises; Mask: code is transformed before model input; Exec.: dynamic confirmation; Repair: patch synthesis. ✓: yes; ✗: no; NR: not reported; n/a: not applicable; F: formal verification instead of execution; P: partial, identifiers only; R: reference harnesses; T: templates.

System	Year	GNN	LLM	Local	Mask	Exec.	Repair
Slither [8]	2019	✗	✗	n/a	n/a	✗	✗
sGuard [21]	2021	✗	✗	n/a	n/a	✗	✓
Smartian [4]	2021	✗	✗	n/a	n/a	✓	✗
ItyFuzz [25]	2023	✗	✗	n/a	n/a	✓	✗
GPTScan [26]	2024	✗	✓	✗	✗	✗	✗
CodeCloak [23]	2025	✗	✓	✗	✓	✗	✗
PromFuzz [15]	2025	✗	✓	NR	✗	✓	✗
PropertyGPT [16]	2025	✗	✓	✗	✗	F	✗
Smart-LLaMA-DPO [34]	2025	✗	✓	✓	✗	✗	✗
LLAMA [10]	2026	✗	✓	NR	✗	✓	✗
MANDO-LLM [20]	2026	✓	✓	NR	✗	✗	✗
This work (PRISM)	2026	✓	✓	✓	P	R	T

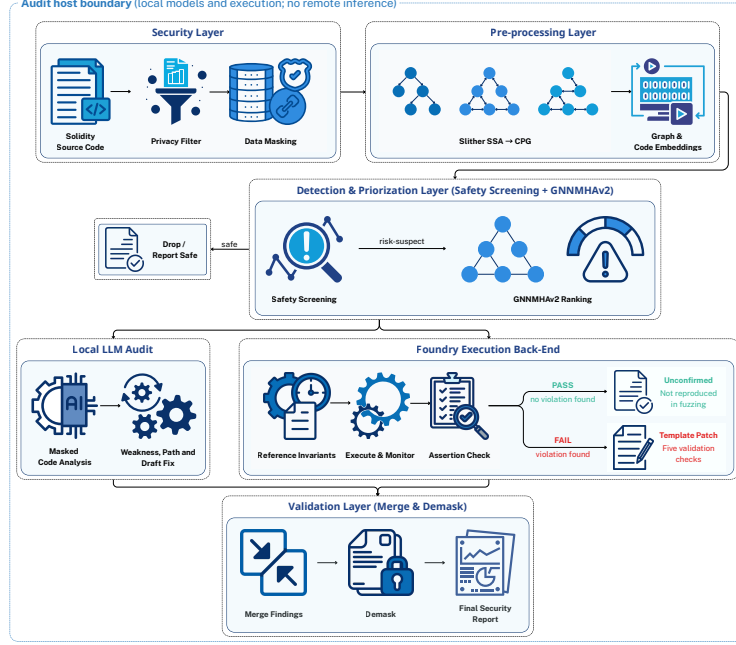


Fig. 1. PRISM architecture. Every stage runs inside the audit host. Routed functions go to two independent branches: a local open-weight LLM, whose findings are reported as unconfirmed hypotheses, and Foundry invariant harnesses, whose findings are reported as confirmed only when an assertion fails. $\mathcal{T}$: pseudonymization map; Φ : routing score; τ : forwarding threshold.

Smart-LLaMA-DPO [34] fine-tunes an 8B open model for explainable detection, which can run locally but does not execute its findings. sGuard [21] and Elysium [9] patch source or bytecode with templates.

Confidential analysis. Provider-side protection of prompts with confidential computing is an active topic [14]. CodeCloak [23] learns prompt manipulations, including identifier replacement, that trade code leakage against suggestion quality for cloud code assistants. On the attack side, ReSym [32] and GenNm [33] recover variable names and types from stripped binaries, showing that renaming does not remove the information that structure carries. PRISM takes the conservative position: inference stays local, and pseudonymization only reduces what artifacts leaving the host reveal.

Table 1 summarizes the capabilities. PRISM is not the strongest detector or fuzzer in isolation; among the systems listed, it is the one that combines local learned inference, artifact masking and execution-based validation.

3 PRISM Design

Figure 1 shows the pipeline; no network access is required once the model weights are installed.

3.1 Threat Model and Pseudonymization

Assets and trust boundary. The asset is unreleased contract source code together with the business logic it reveals: identifier names, constants, addresses and endpoints. The trust boundary is the audit host operated by the contract owner or its auditor.

Adversaries. We consider (A1) third-party model providers and network observers, to which a cloud-LLM workflow would expose code, and (A2) honest-but-curious parties that later see PRISM artifacts outside the host, such as prompt logs, generated harnesses, bug reports shared with external reviewers, or a shared inference server inside the organization. PRISM removes A1 by construction, because no inference is remote; confidentiality against A1 therefore rests on locality, not on masking. DataMasker only reduces what A2 learns from artifacts. A fully compromised audit host is out of scope, because such an adversary can read the plaintext source directly.

Mechanism. PrivacyFilter first removes hard-coded private keys and application programming interface (API) keys, remote procedure call (RPC) endpoints and 20-byte addresses. DataMasker then renames user-defined identifiers of contracts, types, functions and variables, and leaves Solidity keywords, EVM built-ins such as `msg.sender` and `block.timestamp`, standard interfaces such as `IERC20`, numeric and string literals, and comments untouched. The i -th distinct identifier x_i is mapped to

$$\mathcal{T}(x_i) = \text{prism_}\sigma(x_i)_ \langle i \rangle, \quad (1)$$

where $\langle i \rangle$ is a four-digit counter assigned in decreasing order of identifier length and $\sigma(x) \in \{\text{Type}, \text{b}, \text{v}\}$ keeps a coarse lexical role: **Type** for capitalized names, **b** for boolean-style names beginning with `is`, `has` or `can`, and **v** otherwise. $\mathcal{T}$ is a bijection; its inverse $\mathcal{T}^{-1}$ never leaves the host and is applied only to the final report. Both σ and the counter order reveal side information, and Sect. 5.3 measures what $\mathcal{T}$ preserves and what it still reveals to A2.

3.2 Function-Level Code Property Graph

From the pseudonymized source S_{mask} , PRISM extracts Slither’s SSA IR [8] and builds one directed graph per function, $\mathcal{G} = (\mathcal{V}, \mathcal{E}, X, E_{\text{attr}})$, whose nodes are SSA instructions.

- Node features $X \in \mathbb{R}^{N \times 32}$: a 13-dimensional one-hot instruction type, 10 security indicators such as low-level calls, storage reads and writes, guards and transaction-context reads, and 9 structural attributes such as in- and out-degree, loop membership and relative depth.

- Edge attributes $E_{attr} \in \{0, 1\}^{|\mathcal{E}| \times 4}$: a one-hot edge type among `sequential`, `branch`, `loop_back` for control flow and `data_dep` for def-use, so that control and data flow share one edge set.

3.3 Two-Stage Graph Attention Classifier

Both stages use the same encoder, GNNMHA_{v2}, which projects node features to $d = 256$ dimensions and stacks $L = 4$ edge-aware graph attention layers with $K = 8$ heads [3, 28]. With edge attribute $\mathbf{e}_{ij}$ and $\mathcal{N}^+(i) = \mathcal{N}(i) \cup \{i\}$, head k of layer l computes

$$\alpha_{ij}^{k,l} = \text{softmax}_{j \in \mathcal{N}^+(i)} \left(\mathbf{a}^{k,l\top} \text{LeakyReLU}(W_s^{k,l} h_i^l + W_t^{k,l} h_j^l + W_e^{k,l} \mathbf{e}_{ij}) \right), \quad (2)$$

$$h_i^{l+1} = \text{ELU} \left(\text{BN} \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in \mathcal{N}^+(i)} \alpha_{ij}^{k,l} W^{k,l} h_j^l \right) \right) + h_i^l, \quad (3)$$

where BN is batch normalization and ELU the exponential linear unit. Residual connections and a jumping-knowledge (JK) readout, $z_i = [h_i^1 \| h_i^2 \| h_i^3 \| h_i^4] \in \mathbb{R}^{1024}$, keep both local and multi-hop patterns. The graph embedding $g \in \mathbb{R}^{2048}$ concatenates attention pooling and mean pooling,

$$g = \left[\sum_i \beta_i z_i \parallel \frac{1}{N} \sum_i z_i \right], \quad \beta_i = \text{softmax}_i (\mathbf{w}^\top \text{ReLU}(W_p z_i)), \quad (4)$$

and a two-hidden-layer perceptron maps g to class logits. One layer costs $O(K(|\mathcal{V}|d^2 + |\mathcal{E}|d))$, which is linear in graph size for fixed d .

Stage A is a binary model, vulnerable or safe, trained with class-weighted, label-smoothed cross-entropy (CE). Stage B is a five-class model trained with class-weighted focal loss on label-smoothed CE, with smoothing $\epsilon = 0.05$ and $\gamma = 2$:

$$\mathcal{L} = \frac{1}{n} \sum_{m=1}^n \alpha_{y_m} (1 - p_m)^\gamma \text{CE}_\epsilon(g_m, y_m), \quad (5)$$

where p_m is the predicted probability of the true class and α_y the class weight. At inference, Stage A drops functions it predicts safe, Stage B assigns a class to the remaining ones, and PRISM orders them by the Stage-B confidence $\Phi(\mathcal{G}) = \max_c p_B(c \mid \mathcal{G})$, forwarding functions with $\Phi \geq \tau$ ($\tau = 0.3$) in decreasing order of Φ . With five classes $\Phi \geq 0.2$ always, so τ mainly removes near-uniform predictions and the forwarded order is what matters.

3.4 Local LLM Audit, EVM Execution and Repair

Local LLM audit. The masked source and the routed functions, each with its predicted class and Φ , are passed to an open-weight code LLM, DeepSeek-Coder-V2-Lite-Instruct, a 16B-parameter mixture-of-experts model with 2.4B active parameters [37]. It is served on the host by Ollama¹ with temperature 0.1 and at most 4,096 generated tokens. The prompt asks for a structured finding per suspected weakness: weakness type, severity, affected function, attack path, exploit scenario and fix.

¹ <https://ollama.com>; the quantized model occupies 8.3 GiB on disk.

Execution. Independently of the LLM output, PRISM runs a local Foundry back end without network calls. For the evaluated classes, invariant harnesses follow class templates, for example balance conservation for reentrancy and privileged rejection for access control, and are fuzzed locally. In this paper the harnesses are expert-written reference harnesses (Sect. 5.2) and a template replay over the benchmark (Sect. 5.4). An execution finding is reported as confirmed only if an assertion fails with a concrete counterexample trace; LLM findings are reported as unconfirmed hypotheses.

Repair and report. A template-based repair module proposes a patch for each confirmed finding, which must pass five checks (Sect. 5.5). LLM hypotheses and execution results are merged and deduplicated by function and weakness type, keeping their confirmation status, and identifiers are restored through $\mathcal{T}^{-1}$ only in the final report.

4 Experimental Setup

Research questions (RQs). RQ1: How accurately does the classifier label vulnerable functions on seen and unseen contract lineages, and how does it compare with other graph learners? RQ2: Does the local EVM back end execute harnesses reliably and reproducibly? RQ3: Does pseudonymization preserve analysis utility, and what does it still reveal? RQ4: How far do taxonomy- and severity-aware filters reduce alarms on safe contracts without losing vulnerable ones? RQ5: Do template patches block exploits while preserving interfaces?

Dataset. We extract 12,991 function-level CPGs with at least three nodes from three public sources: SmartBugs Wild [7] (1,179 graphs), the SolidiFI benchmark [11] (8,105 graphs) and LISA-Bench² (3,707 graphs), a collection of professional audit findings. The graphs cover five Smart Contract Weakness Classification (SWC) classes: Reentrancy (SWC-107, 2,475), Integer Overflow (SWC-101, 2,392), Access Control (SWC-105, 2,026), Unchecked Return Value (SWC-104, 2,983) and Timestamp or Transaction-Order Dependence (SWC-114, 3,115). Graphs of the remaining source categories, denial of service and delegatecall, are excluded from the five-class task. Labels follow a contract-derived weak-supervision policy: SmartBugs and SolidiFI functions inherit the label of their contract, and LISA-Bench functions inherit the label obtained by keyword mapping of the finding text. We audited this choice on SolidiFI: of 9,369 injected bug spans in its 350 BugLog contracts, 4,947 map uniquely to a function body, and a strictly span-mapped subset would keep only 2,580 graphs with two Reentrancy and two timestamp dependence examples, too few for a five-class task. Stage A is trained on a larger binary corpus of 24,454 graphs that also contains the excluded categories and graphs labeled safe, with labels 0–7 treated as vulnerable and label 8 as safe.

² AgentLISA: LISA-Bench: Smart contract vulnerability detection benchmark. <https://github.com/agentlisa/bench> (2025), accessed September 2026.

Table 2. Comparison with graph baselines on both splits, five-seed mean $\pm$ std. Single-seed GAT and GATv2 score within 0.5 points of GNNMHA_{v2} (0.7562 and 0.7571). Slither 0.11.5 is mapped to the five classes. Best mean per column in bold.

Method	Random ($n = 1,949$)		Lineage-disjoint ($n = 2,146$)	
	Accuracy	Macro-F1	Accuracy	Macro-F1
Slither (mapped)	0.2299	0.0748	0.2330	0.0756
GCN	0.7434 $\pm$.0016	0.7604 $\pm$.0022	0.7445 $\pm$.0008	0.7630 $\pm$.0009
GIN	0.7442 $\pm$.0005	0.7609 $\pm$.0005	0.7432 $\pm$.0011	0.7617 $\pm$.0011
GNNMHA _{v2} (ours)	0.7426 $\pm$.0014	0.7595 $\pm$.0019	0.7445 $\pm$.0006	0.7631 $\pm$.0006

Splits. We use two frozen protocols for the five-class classifier. The random split is stratified 70/15/15 (9,093/1,949/1,949 graphs). The lineage-disjoint split partitions contracts by origin so that SolidiFI `buggy_N` variants, functions from the same LISA-Bench finding, and functions from the same imported source contract never cross partitions, giving 8,349/2,496/2,146 graphs with zero lineage overlap. Stage A uses a frozen binary 70/15/15 split (17,118/3,667/3,669; test: 3,132 vulnerable, 537 safe).

Baselines. To isolate the effect of the encoder, we train graph convolutional (GCN) and graph isomorphism (GIN) networks as well as GAT and GATv2 on the same graphs, node features, splits and training budget; GAT and GATv2 do not use edge attributes. As a rule-based reference, Slither 0.11.5 is mapped to the five classes through its detector names, for example `reentrancy-eth` to Reentrancy and `unchecked-lowlevel` to Unchecked Return, and a function that triggers no mapped detector is assigned the majority training class.

Implementation. Models are implemented in PyTorch Geometric and trained with AdamW (learning rate 10^{-3} , weight decay 10^{-4}), batch size 64 and OneCycleLR for up to 35 epochs with early stopping on validation macro-F1 (patience 15). GNNMHA_{v2}, GCN and GIN are trained with five seeds (42 to 46); GAT and GATv2 with seed 42. Experiments run on an Apple M4 CPU with 10 cores and 16 GB RAM. One 35-epoch training run of GNNMHA_{v2} takes 348.3 ± 37.2 s, compared with 56.5 ± 6.7 s for GCN and 67.4 ± 1.3 s for GIN, and inference takes 0.21 ms per function graph. Execution uses Foundry Forge 1.5.1 and `solc` 0.8.30.

5 Results

5.1 RQ1: Function-Level Classification Accuracy

Table 2 compares the classifier with graph baselines. GNNMHA_{v2} reaches 0.7595 ± 0.0019 macro-F1 on the random split and 0.7631 ± 0.0006 on the lineage-disjoint split, with macro-precision 0.887 ± 0.005 and macro-recall 0.725 ± 0.001

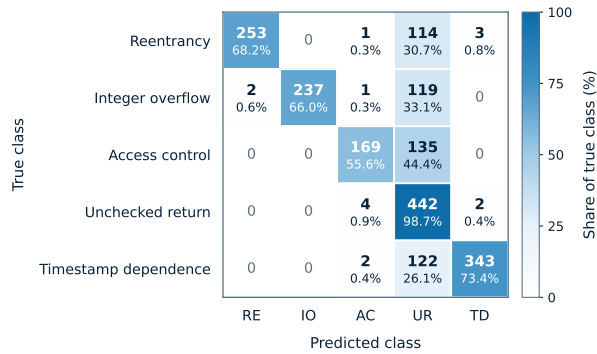


Fig. 2. Confusion matrix of a single GNNMHA v2 training run on the frozen random split (seed 42; $n = 1,949$; accuracy 0.741, macro-F1 0.758; five-seed results are in Table 2). Cells show counts and the share of the true class. Columns follow the row order: Reentrancy (RE, SWC-107), Integer overflow (IO, SWC-101), Access control (AC, SWC-105), Unchecked return (UR, SWC-104), Timestamp dependence (TD, SWC-114).

on the random split; macro-F1 exceeds accuracy because the large Unchecked Return class has low precision while the smaller classes are predicted precisely. The mapped Slither reference scores 0.0748 because its detectors fire on few function slices under our label mapping. Over five fresh seeds, Stage A reaches $88.62 \pm 0.24\%$ accuracy, with $87.82 \pm 0.32\%$ vulnerable recall and $93.26 \pm 0.61\%$ safe specificity ($n = 3,669$).

The more informative result is that all graph learners converge to the same level. On both splits, GCN, GIN, GAT, GATv2 and GNNMHA v2 lie within 0.5 macro-F1 points of each other. In 20 paired exact McNemar tests between GNNMHA v2 and GCN or GIN over two splits and five seeds, no difference is significant even without multiple-comparison correction (all $p > 0.16$), the paired models disagree on at most 22 test functions per run, and between 97.4% and 99.6% of GNNMHA v2’s errors are shared with the baseline. Pairwise Jaccard error overlap exceeds 96.5%, and 493 of 520 errors (94.8%) are shared across all five architectures. When models of different design make the same errors, label noise is a more plausible limit than encoder capacity. A structural-label-conflict diagnostic supports this: 6.2% of random-split test graphs match a training graph with a conflicting label because of contract-level inheritance; on these graphs accuracy falls to 7.9%, whereas on conflict-free graphs it reaches 78.7%.

The per-class view in Fig. 2 shows where the shared errors go. Precision is 0.95 to 1.00 for the four specific classes, but their recall is 0.56 to 0.73, and 490 of the 505 errors are predictions of Unchecked Return (precision 0.47, recall 0.99), whose SSA signature, an external call followed by an unused result, is the most generic. The model therefore rarely confuses two specific weaknesses. It assigns functions that carry no class-specific evidence to the most generic class, which is what contract-derived labels produce when a contract’s label is inherited by functions that do not contain the flaw. Consistently, five-seed top-2 accuracy is

Table 3. Ablation of GNNMHA_{v2} on the random split, five seeds, mean $\pm$ std. Δ : change in mean macro-F1 relative to the full model.

Variant	Macro-F1	Δ	Top-2 accuracy
Full model	0.7595 ± 0.0019	–	0.8086
without edge attributes	0.7603 ± 0.0011	+0.0008	0.8085
without jumping knowledge	0.7597 ± 0.0018	+0.0002	0.8117
focal loss $\rightarrow$ cross-entropy	0.7584 ± 0.0012	–0.0010	0.8097
attention+mean $\rightarrow$ mean pooling	0.7596 ± 0.0029	+0.0001	0.8115
$L = 2$	0.7611 ± 0.0016	+0.0016	0.8079
$L = 3$	0.7588 ± 0.0025	–0.0007	0.8069
$K = 4$	0.7588 ± 0.0017	–0.0007	0.8089

0.8086, so the second-ranked class recovers only 25.6% of the errors: most errors are not near misses.

The ablation in Table 3 supports the same reading. Removing edge attributes, jumping knowledge or attention pooling, replacing focal loss, and changing depth or head count all move macro-F1 by at most 0.16 points, within one to two standard deviations of seed variation. Under contract-derived labels, encoder design is not the bottleneck; label granularity is. For routing, any of the tested encoders orders functions at the same quality and negligible cost, so better labels, not a larger encoder, are the lever for improvement.

5.2 RQ2: Execution Back-End Reliability

We executed expert-written reference harnesses for five case-study contracts under 30 fuzzing seeds (101 to 130) in a directed exploit test and a 10,000-call fuzzing campaign, for $5 \times 30 \times 2 = 300$ runs. The targets are SimpleDAO (SWC-107), TokenSale (SWC-101), UnprotectedVault (SWC-105), Unchecked-Bank (SWC-104) and TimestampLock (SWC-114). All 300 runs passed without timeouts, with mean directed and fuzzing latencies of 190.26 and 202.75 ms, and directed transaction gas was deterministic at 77,558, 39,648, 48,550, 27,202 and 39,827, respectively. A seed-42 integrated trace ran masking, preprocessing, GNN routing, a bounded local LLM audit and the reference tests for all five cases, and every stage completed. This shows that the local stages run together reproducibly; because the harnesses are hand-written, it does not measure how often generated harnesses find bugs.

5.3 RQ3: Pseudonymization Fidelity and Residual Exposure

Utility. Table 4 shows that DataMasker is lossless for compilation and restoration: every masked contract compiles with `solc`, and $\mathcal{T}^{-1}$ restores every contract exactly. For static analysis, all 229 contract-level Slither detector types raised on plaintext code are preserved on masked code, with six further types added by naming conventions. For the LLM stage, we ran a taxonomy-guided audit

Table 4. DataMasker on the 57 benchmark contracts with 286 user-defined identifiers. Upper block: utility, higher is better. Lower block: exposure to an A2 adversary, lower is better.

Metric	Value	Meaning
Compilation validity	57/57 (100%)	masked code compiles
Exact restoration	57/57 (100%)	$\mathcal{T}^{-1}$ restores the source
Slither detector types retained	229/229 (100%)	per contract; 6 added
Dictionary exposure	87/286 (30.4%)	name in a 32-term domain list
Frequency re-linkage	256/286 (89.5%)	rank matching of counts

over the five SWC classes without GNN hotspots. On raw code it recalls 30/30 vulnerable contracts with 11/27 (40.7%) safe specificity, so unconstrained inspection raises many false alarms on benign patterns. On pseudonymized code, recall drops to 13/30 (43.3%) while safe specificity rises to 19/27 (70.4%): once role-revealing names such as `owner` are masked, a text-only audit loses most of its cues. For weaknesses expressed through EVM built-ins such as `block.timestamp` and external `.call`, raw and masked audits give identical predictions (Jaccard $J = 1.00$). Masking thus costs recall exactly where the model relies on identifier semantics, which motivates structural hotspots from the graph classifier.

Exposure. We quantify two A2 adversaries. The dictionary measure counts identifiers whose plaintext name is one of 32 common Solidity and DeFi terms such as `owner`, `balance` and `withdraw`; an adversary who guesses these terms from the role of a pseudonym recovers at most 30.4% of identifiers. The frequency adversary knows how often each plaintext identifier occurs in the contract, for example from an earlier version or a leaked excerpt, ranks identifiers and pseudonyms by occurrence count, and aligns the two rankings. Because $\mathcal{T}$ renames one-to-one and preserves occurrence counts, only ties stop it, and it re-links 89.5% of identifiers. This upper bound assumes strong side knowledge, but it shows that renaming preserves code structure, so pseudonymization alone is not a confidentiality mechanism for code sent to an untrusted model [23, 32, 33]; it is defense in depth for artifacts that leave the host.

5.4 RQ4: Alarm Reduction on Safe Contracts

We ran Slither 0.11.5 on the 57-contract benchmark, with 27 safe and 30 vulnerable contracts labeled by their benchmark templates, and applied PRISM’s static filters in sequence (Table 5).

Unfiltered Slither raises 230 alarms and flags every contract, which gives 0/27 specificity and 30/57 (52.6%) contract-level precision, mostly because of stylistic detectors. Restricting output to the five target classes (Stage 1) clears 21 of 27 safe contracts (77.8% specificity); 12 vulnerable contracts lack a dedicated Slither detector under this mapping and are no longer flagged. Filtering for high or medium impact and confidence (Stage 2) removes eight secondary alarms on already-flagged contracts, keeping specificity at 21/27 and all 18 detected flaws.

Table 5. Alarms along the static filtering cascade on 57 contracts. Safe cleared measures contract-level specificity; Vuln. retained counts vulnerable contracts that still raise an alarm.

Stage	Filter	Alarms	Safe cleared (↑)	Vuln. retained
0	Slither, all detectors	230	0/27 (0.0%)	30/30 (100%)
1	Five target SWC classes only	41	21/27 (77.8%)	18/30 (60.0%)
2	High/medium impact, confidence	33	21/27 (77.8%)	18/30 (60.0%)

We then replayed the class templates in Foundry on all 57 labeled fixtures (20.90 s under `solc` 0.8.30): the 30 vulnerable fixtures satisfy their exploit predicates and the 27 safe fixtures satisfy their safety predicates. The six safe contracts that still raise a Stage-2 alarm are among these 27, so execution refutes all six residual false alarms.

5.5 RQ5: Patch Validation

For the five case-study contracts with exploit harnesses (Sect. 5.2), template repair produced one patch each. Each patch passed five checks: (1) compilation with `solc` 0.8.30; (2) minimal change, at most 25% of lines measured on the abstract syntax tree; (3) the original exploit harness now reverts; (4) the 10,000-call fuzzing campaign still passes; and (5) function selectors and storage layout are unchanged, which preserves the external application binary interface and proxy storage layout.

6 Discussion and Limitations

Scope of the evidence. Graph screening costs 0.21 ms per function, negligible next to one Foundry run of about 0.2 s, and the LLM is invoked only for functions with $\Phi \geq \tau$. The stages are evaluated separately. We do not report within-contract ranking metrics such as recall at k , a joint accuracy for the Stage-A and Stage-B cascade, or the recall of the masked LLM when its prompt includes GNN hotspots; the last is the measurement that would show whether structural routing recovers the recall that masking removes in Sect. 5.3. Execution and repair are evaluated on five single-contract cases with expert-written harnesses, so they establish feasibility and determinism rather than bug-finding frequency, and automated harness synthesis for multi-file mainnet contracts remains open. The encoder comparison covers generic graph learners; contract-specific models such as MANDO-LLM [20] and fine-tuned local LLMs [34] were not retrained on our labels.

Threats to validity. Construct validity: function graphs inherit contract-derived labels, so benign functions are counted as positives, which bounds classification accuracy; the 57 benchmark labels follow injection templates rather than manual

spans, and SWC-114 targets are timestamp proxies. Internal validity: DataMasker leaves comments and string literals in clear, and NatSpec comments can describe business logic; its counter order and role prefix leak length rank and lexical role, which a keyed random permutation would remove. Slither IR extraction may degrade on inline Yul. External validity: the benchmark contracts are small reference implementations under `solc` 0.8.30, and transfer across data sources was not evaluated.

Data availability. Code, configurations, frozen split manifests, result files and reference harnesses are available in an anonymized repository: <https://anonymous.4open.science/r/PRISM-PROJECT/>.

7 Conclusion

This paper presented PRISM, a smart contract audit pipeline that keeps every model and artifact on the audit host and separates LLM hypotheses from execution-confirmed findings. Its two-stage graph attention classifier routes functions at $76.31 \pm 0.06\%$ macro-F1 on unseen contract lineages in 0.21 ms per function, and the agreement of generic graph encoders on the same errors shows that contract-derived labels, not model capacity, limit function-level accuracy. Reversible pseudonymization preserves compilation and static analysis, yet a frequency adversary re-links 89.5% of identifiers, so confidentiality has to come from local inference. Static filtering reduces flagged safe contracts from 27/27 to 6/27, execution refutes the six residual alarms, and reference harnesses and template patches run deterministically on commodity hardware.

Future work will measure the masked LLM audit with GNN hotspots and the full cascade end to end on larger production benchmarks, replace contract-derived supervision with function-level ground truth, mask comments and randomize pseudonym indices, and extend template repair to automated harness and patch synthesis.

References

1. Alsunaidi, S.J., Aljamaan, H., Hammoudeh, M.: Leveraging machine learning models to improve smart contract security: A survey of vulnerabilities and detection methods. *ACM Computing Surveys* **58**(6), 1–37 (2025)
2. Atzei, N., Bartoletti, M., Cimoli, T.: A survey of attacks on Ethereum smart contracts (SoK). In: *Principles of Security and Trust (POST)*. pp. 164–186 (2017)
3. Brody, S., Alon, U., Yahav, E.: How attentive are graph attention networks? In: *International Conference on Learning Representations (ICLR)* (2022)
4. Choi, J., Kim, D., Kim, S., Grieco, G., Groce, A., Cha, S.K.: Smartian: Enhancing smart contract fuzzing with static and dynamic data-flow analyses. In: *Proc. 36th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*. pp. 227–239 (2021)
5. Croft, R., Babar, M.A., Kholoosi, M.M.: Data quality for software vulnerability datasets. In: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. pp. 121–133. IEEE (2023)

6. Ding, Y., Fu, Y., Ibrahim, O., Sitawarin, C., Chen, X., Alomair, B., Wagner, D., Ray, B., Chen, Y.: Vulnerability detection with code language models: How far are we? arXiv preprint arXiv:2403.18624 (2024)
7. Durieux, T., Ferreira, J.F., Abreu, R., Cruz, P.: Empirical review of automated analysis tools on 47,587 Ethereum smart contracts. In: Proc. 42nd IEEE/ACM Int. Conf. Software Engineering (ICSE). pp. 530–541 (2020)
8. Feist, J., Grieco, G., Groce, A.: Slither: a static analysis framework for smart contracts. In: Proc. 2nd Int. Workshop Emerging Trends in Software Engineering for Blockchain (WETSEB). pp. 8–15 (2019)
9. Ferreira Torres, C., Jonker, H., State, R.: Elysium: Context-aware bytecode-level patching to automatically heal vulnerable smart contracts. In: Proc. 25th Int. Symp. Research in Attacks, Intrusions and Defenses (RAID). pp. 115–128 (2022)
10. Gai, K., Liang, H., Yu, J., Zhu, L., Niyato, D.: LLAMA: Multi-feedback smart contract fuzzing framework with LLM-guided seed generation. IEEE Trans. Information Forensics and Security (2026)
11. Ghaleb, A., Pattabiraman, K.: How effective are smart contract analysis tools? evaluating smart contract static analysis tools using bug injection. In: Proc. 29th ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA). pp. 415–427 (2020)
12. Grieco, G., Song, W., Cygan, A., Feist, J., Groce, A.: Echidna: effective, usable, and fast fuzzing for smart contracts. In: Proc. 29th ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA). pp. 557–560 (2020)
13. Jiang, B., Liu, Y., Chan, W.K.: ContractFuzzer: Fuzzing smart contracts for vulnerability detection. In: Proc. 33rd IEEE/ACM Int. Conf. Automated Software Engineering (ASE). pp. 259–269 (2018)
14. Li, C., Gim, I., Zhong, L.: Confidential prompting: Privacy-preserving LLM inference on cloud. arXiv preprint arXiv:2409.19134 (2024)
15. Lin, X., Xie, Q., Zhao, B., Tian, Y., Zonouz, S., et al.: PromFuzz: Leveraging LLM-driven and bug-oriented composite analysis for detecting functional bugs in smart contracts. In: Proc. 40th IEEE/ACM Int. Conf. Automated Software Engineering (ASE). pp. 1044–1056 (2025)
16. Liu, Y., Xue, Y., Wu, D., Sun, Y., Li, Y., Shi, M., Liu, Y.: Propertygpt: Llm-driven formal verification of smart contracts through retrieval-augmented property generation. arXiv preprint arXiv:2405.02580 (2024)
17. Liu, Z., Qian, P., Wang, X., Zhuang, Y., Qiu, L., Wang, X.: Combining graph neural networks with expert knowledge for smart contract vulnerability detection. IEEE Trans. Knowledge and Data Engineering **35**(2), 1296–1310 (2023)
18. Luu, L., Chu, D.H., Olickel, H., Saxena, P., Hobor, A.: Making smart contracts smarter. In: Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS). pp. 254–269 (2016)
19. Ma, W., Wu, D., Sun, Y., Wang, T., Liu, S., et al.: Combining fine-tuning and LLM-based agents for intuitive smart contract auditing with justifications. In: Proc. 47th IEEE/ACM Int. Conf. Software Engineering (ICSE). pp. 1742–1754 (2025)
20. Nguyen, N.M., Nguyen, H.H., Long Le, T., Ahmadi, Z., Doan, T.N., et al.: MANDO-LLM: Heterogeneous graph transformers with large language models for smart contract vulnerability detection. ACM Trans. Software Engineering and Methodology **35**(6), 1–30 (2026)
21. Nguyen, T.D., Pham, L.H., Sun, J.: sGuard: Towards fixing vulnerable smart contracts automatically. In: IEEE Symp. Security and Privacy (S&P). pp. 1215–1229 (2021)

22. Nguyen, T.D., Pham, L.H., Sun, J., Lin, Y., Minh, Q.T.: sFuzz: An efficient adaptive fuzzer for Solidity smart contracts. In: Proc. 42nd IEEE/ACM Int. Conf. Software Engineering (ICSE). pp. 778–788 (2020)
23. Noah, A.F., Shapira, A., Kochva, E.B., Maimon, I., Mimran, D., Elovici, Y., Shabtai, A.: Codecloak: A method for mitigating code leakage by llm code assistants. In: ECAI 2025: 28th European Conference on Artificial Intelligence 25–30 October 2025, Bologna, Italy-Including 14th Conference on Prestigious Applications of Intelligent Systems (PAIS 2025) Proceedings. pp. 4418–4427. SAGE Publications 1 Oliver’s Yard, 55 City Road, London, EC1Y 1SP (2025)
24. Pang, Y., Liu, X., Huang, T., Hong, Y., Huang, J., et al.: Graph-based contract sensing framework for smart contract vulnerability detection. *IEEE Trans. Big Data* (2025)
25. Shou, C., Tan, S., Sen, K.: ItyFuzz: Snapshot-based fuzzer for smart contract. In: Proc. 32nd ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA). pp. 322–333 (2023)
26. Sun, Y., Wu, D., Xue, Y., Liu, H., Wang, H., et al.: GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis. In: Proc. 46th IEEE/ACM Int. Conf. Software Engineering (ICSE). pp. 1–13 (2024)
27. Tsankov, P., Dan, A., Drachsler-Cohen, D., Gervais, A., Buenzli, F., Vechev, M.: Securify: Practical security analysis of smart contracts. In: Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS). pp. 67–82 (2018)
28. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y.: Graph attention networks. In: International Conference on Learning Representations (ICLR) (2018)
29. Wang, S.J., Pei, K., Yang, J.: SmartInv: Multimodal learning for smart contract invariant inference. In: IEEE Symp. Security and Privacy (S&P). pp. 2217–2235 (2024)
30. Wei, Z., Sun, J., Sun, Y., Liu, Y., Wu, D., Zhang, Z., Zhang, X., Li, M., Liu, Y., Li, C., et al.: Advanced smart contract vulnerability detection via llm-powered multi-agent systems. *IEEE Transactions on Software Engineering* (2025)
31. Wüstholtz, V., Christakis, M.: Harvey: a greybox fuzzer for smart contracts. In: Proc. 28th ACM Joint European Software Engineering Conf. and Symp. Foundations of Software Engineering (ESEC/FSE). pp. 1398–1409 (2020)
32. Xie, D., Zhang, Z., Jiang, N., Xu, X., Tan, L., Zhang, X.: ReSym: Harnessing LLMs to recover variable and data structure symbols from stripped binaries. In: Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS). pp. 4554–4568 (2024)
33. Xu, X., Zhang, Z., Su, Z., Huang, Z., Feng, S., et al.: Unleashing the power of generative model in recovering variable names from stripped binary. In: Network and Distributed System Security Symposium (NDSS) (2025)
34. Yu, L., Huang, Z., Yuan, H., Cheng, S., Yang, L., Zhang, F., Shen, C., Ma, J., Zhang, J., Lu, J., et al.: Smart-llama-dpo: Reinforced large language model for explainable smart contract vulnerability detection. *Proceedings of the ACM on Software Engineering* **2**(ISSTA), 182–205 (2025)
35. Zhen, Z., Zhao, X., Zhang, J., Wang, Y., Chen, H.: DA-GNN: a smart contract vulnerability detection method based on dual attention graph neural network. *Computer Networks* **242**, 110238 (2024)
36. Zhu, H., Yang, L., Wang, L., Sheng, V.S.: A survey on security analysis methods of smart contracts. *IEEE Trans. Services Computing* **17**(6), 4522–4539 (2024)

- 37. Zhu, Q., Guo, D., Shao, Z., Yang, D., Wang, P., et al.: DeepSeek-Coder-V2: Breaking the barrier of closed-source models in code intelligence. arXiv preprint arXiv:2406.11931 (2024)
- 38. Zhuang, Y., Liu, Z., Qian, P., Liu, Q., Wang, X., He, Q.: Smart contract vulnerability detection using graph neural network. In: Proc. 29th Int. Joint Conf. Artificial Intelligence (IJCAI). pp. 3283–3290 (2020)